SISTEMAS DIGITALES · Unidad 1: Microcontroladores para sistemas embebidos · Semana 2 · Sesión 1

Del AVR al ESP32

Pines, registros y el programa que nunca termina

16 17 18 19 21 3V3

Ing. Luis Sinche Cueva, Mg. · Carrera de Computación – UNL · Octubre 2026

Objetivo de la sesión

Al finalizar, el estudiante podrá:

1. Relacionar los registros GPIO del ESP32 con DDR, PORT y PIN del AVR.
2. Elegir pines seguros del ESP32 DevKit V1 y justificar las restricciones eléctricas y de arranque.
3. Explicar cómo arranca y se ejecuta un programa embebido (setup, loop y superlazo).

Resultado de aprendizaje

R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.

Activación: ¿qué hace cada línea?

```
DDRB  |= (1 << PB5);      // ①  ¿?
PORTB |= (1 << PB5);      // ②  ¿?
if (PIND & (1 << PD2))    // ③  ¿?
```

Recuerden la semana 1

- **DDR** — dirección: ¿entrada o salida?
- **PORT** — salida: ¿alto o bajo?
- **PIN** — entrada: ¿qué llega?

Pregunta para hoy: si el ESP32 no tiene DDRB, PORTB ni PINB… ¿se pierde todo lo aprendido la semana pasada?

1

Parte 1 · Del AVR al ESP32

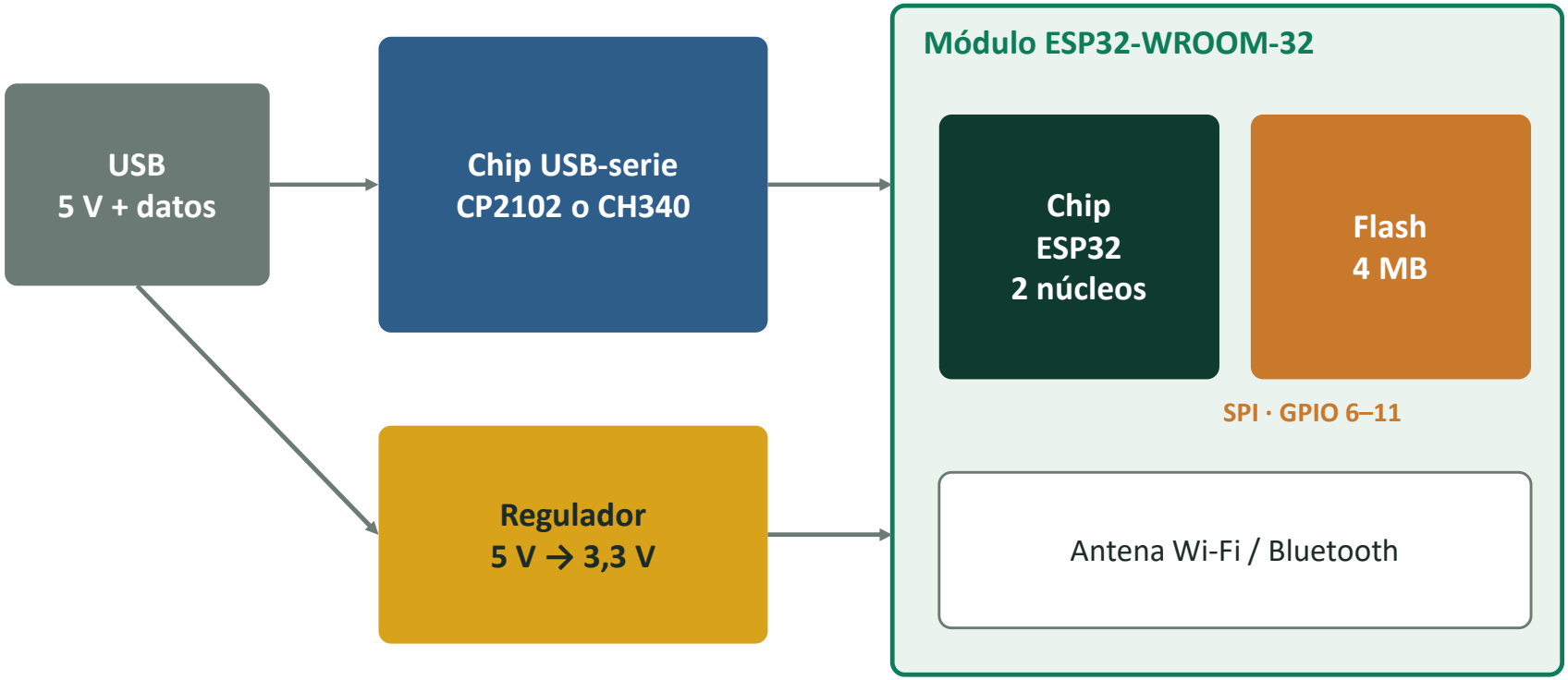
La misma idea, otros nombres, 32 bits

De la tienda de barrio al supermercado

	ATmega328P (semana 1)	ESP32-WROOM-32
Ancho de palabra	8 bits	32 bits
Núcleos / reloj	1 núcleo · 16 MHz	2 núcleos Xtensa LX6 · hasta 240 MHz
SRAM	2 KB	520 KB
Flash	32 KB, dentro del chip	4 MB, chip aparte en el módulo (SPI)
Tensión lógica	5 V	3,3 V
Pin ↔ periférico	fijo (PB5 = bit 5 de PORTB)	configurable (GPIO matrix)

Lo que NO cambia: un pin se controla escribiendo bits en registros mapeados en memoria.

La placa DevKit V1 por dentro



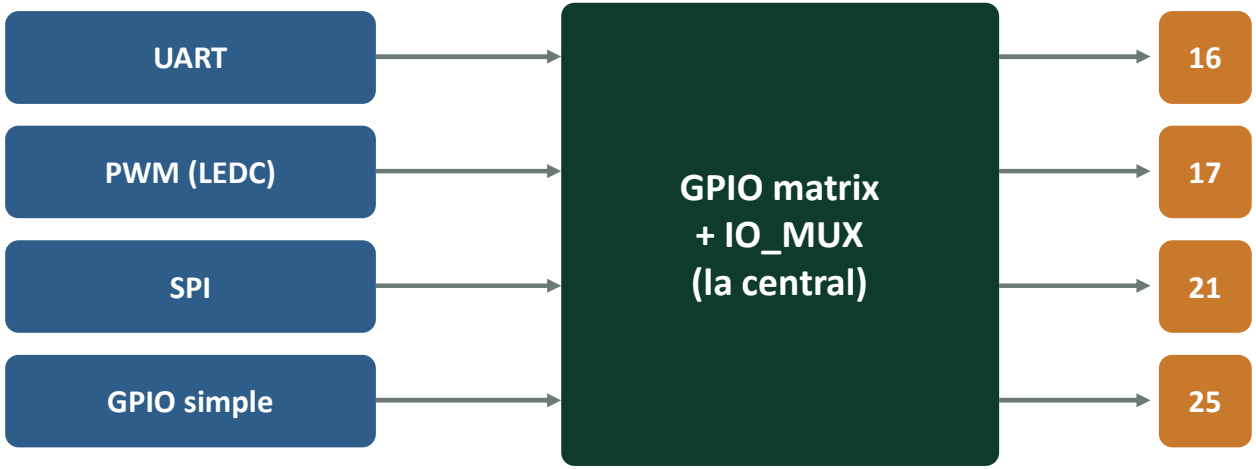
Usaremos el DevKit V1 de 30 pines. Verifiquen la serigrafía: los clones cambian el orden de pines.

Entre pines y periféricos hay una central

AVR: cableado fijo



ESP32: central configurable



Configurar un pin = 3 pasos:

1. Elegir su función (IO_MUX)
2. Conectar la señal en la matrix
3. Habilitar la salida

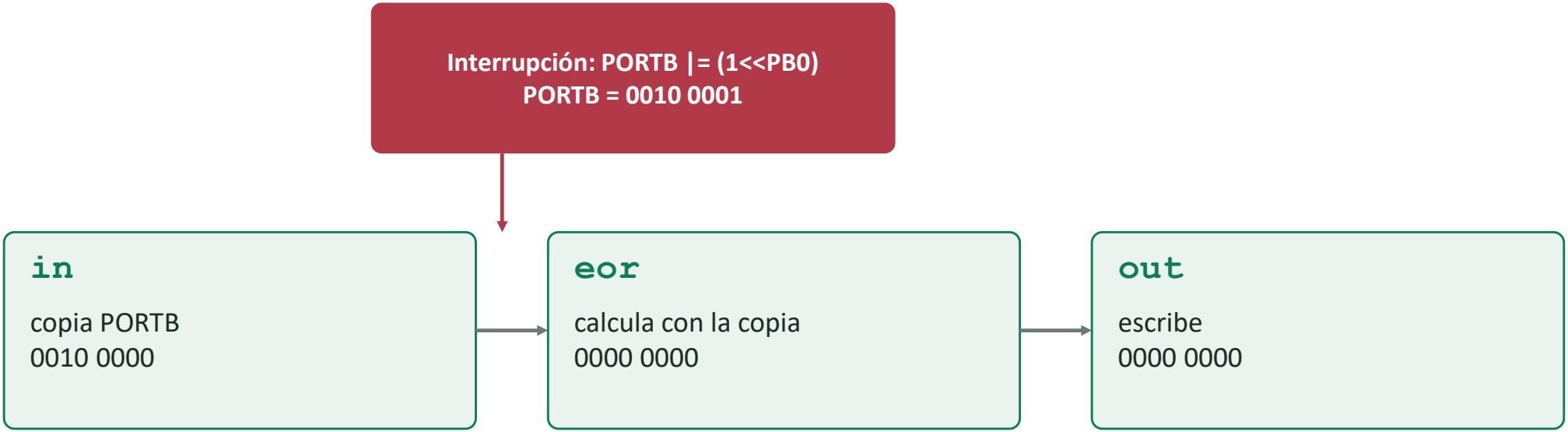
pinMode() hace los tres por nosotros.

DDR, PORT y PIN tienen equivalentes

Función	AVR (semana 1)	ESP32 (GPIO 0–31)
Dirección (entrada/salida)	`DDRB`	`GPIO_ENABLE_REG`
Escribir la salida	`PORTB`	`GPIO_OUT_W1TS_REG / GPIO_OUT_W1TC_REG`
Leer la entrada	`PINB`	`GPIO_IN_REG`
Pull-up interna	`PORTB (con el pin como entrada)`	`registro IO_MUX de cada pin`

GPIO 32–39 no caben en un registro de 32 bits: usan un segundo banco (GPIO_OUT1_REG, GPIO_IN1_REG).

El riesgo de leer, modificar y escribir



Resultado: PORTB = 0000 0000 → el cambio de la interrupción se perdió. Con dos núcleos el riesgo es mayor.

W1TS y W1TC: notas de "encender" y "apagar"

W1TS · write 1 to set

Bits en 1 → se ENCIENDEN
Bits en 0 → no se tocan

W1TC · write 1 to clear

Bits en 1 → se APAGAN
Bits en 0 → no se tocan

GPIO_OUT antes	0 1 0 1
escribo en W1TS	0 0 1 0
GPIO_OUT después	0 1 1 1

Una sola escritura, sin leer antes → la operación es atómica.

Ejemplo: Blink con registros del ESP32

```
#include "soc/gpio_reg.h"

constexpr uint8_t  PIN_LED  = 16;
constexpr uint32_t MASK_LED = 1UL << PIN_LED;

void setup() {
  pinMode(PIN_LED, OUTPUT);              // configura
}

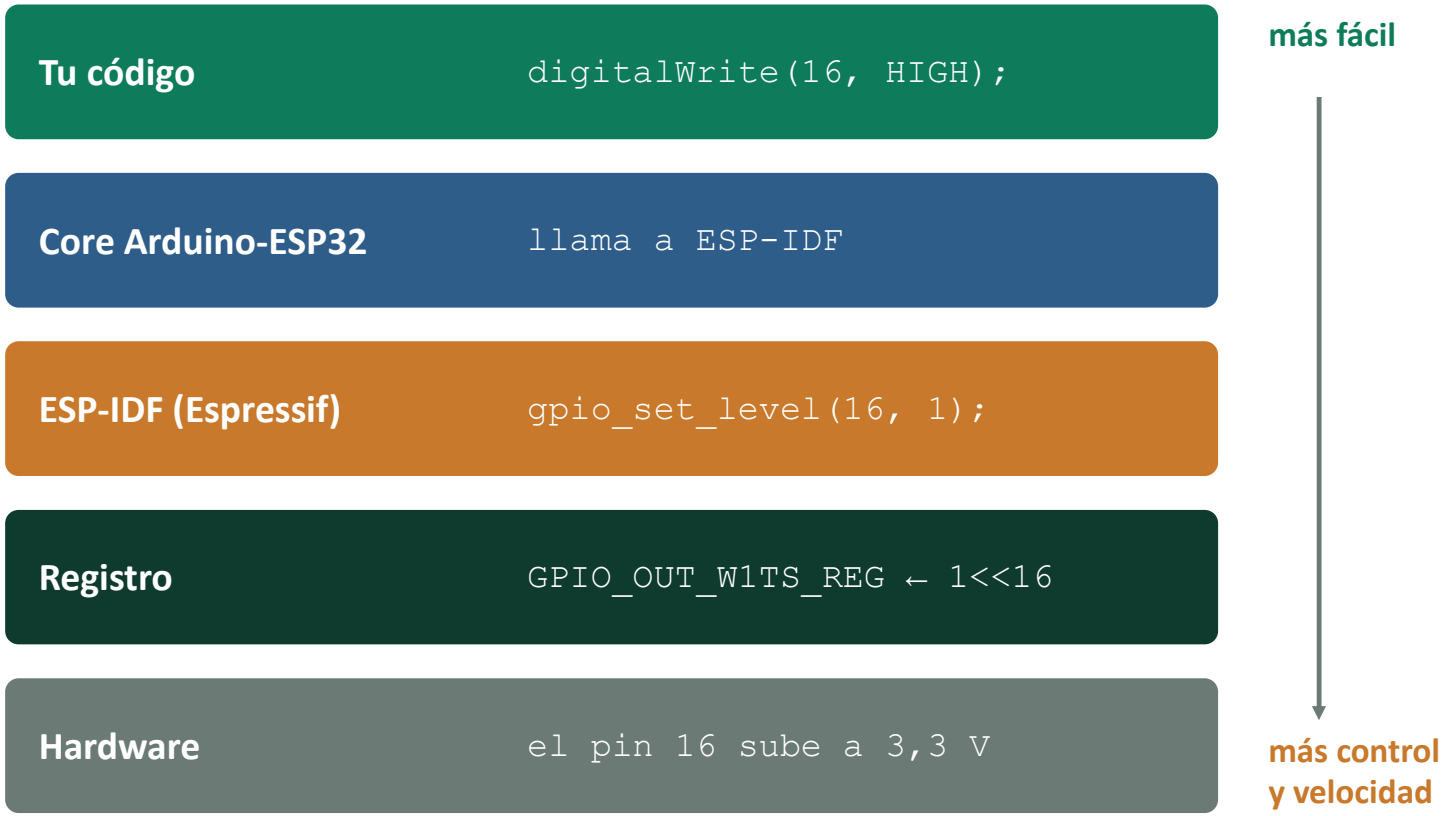
void loop() {
  REG_WRITE(GPIO_OUT_W1TS_REG, MASK_LED); // PORTB |=
  delay(500);
  REG_WRITE(GPIO_OUT_W1TC_REG, MASK_LED); // PORTB &= ~
  delay(500);
}
```

Montaje

LED en GPIO16
330 Ω a GND
Placa: ESP32 Dev Module
Core Espressif 3.x

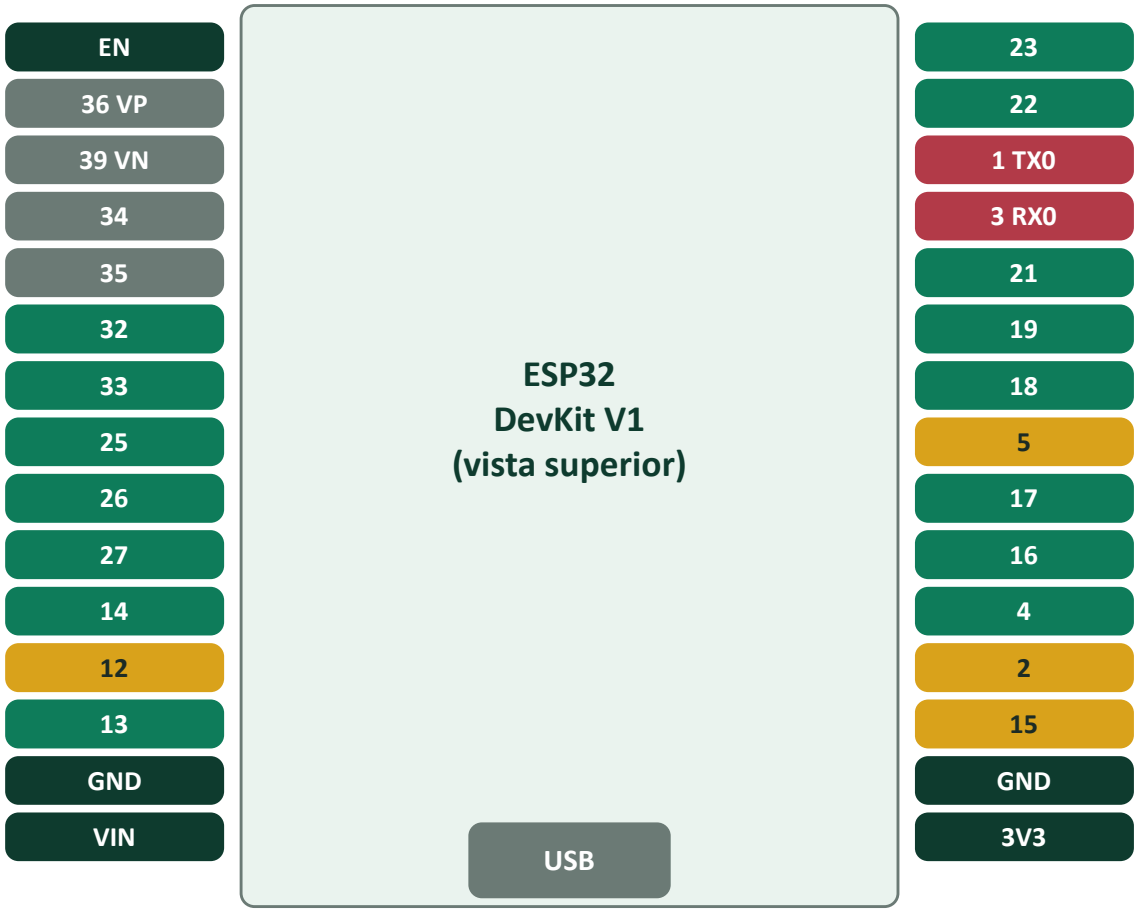
pinMode() configura.
Los registros leen y escriben.

Capas de abstracción



Recuerden la Parte D de la AP1: la API cuesta tiempo.

Mapa de pines del DevKit V1 (30 pines)



Seguros para LEDs y pulsadores

Strapping: el chip los lee al arrancar

Solo entrada, sin pull-up interna

UART0: carga y Monitor Serie

Alimentación y reset

AP2: LEDs en 16, 17, 18, 19 y pulsador en 21.
Los pines 6–11 (flash) no salen en esta placa.

Pines de strapping: el chip los mira al despertar

GPIO0

En BAJO al arrancar → modo de carga.
Es el botón BOOT.

GPIO2

Debe estar en bajo o al aire para cargar.
Suele tener el LED azul de la placa.

GPIO12

En ALTO al arrancar → flash a 1,8 V.
La placa no arranca.

GPIO5 y GPIO15 también se leen al arrancar.
Regla práctica: no conectar entradas a pines de strapping.

Wokwi NO reproduce estas fallas: solo aparecen con la placa real.

3,3 V: los límites eléctricos

$$R = (3{,}3\ V - Vf) / I$$

LED rojo, 330 Ω → (3,3 − 2,0) / 330 ≈ 4 mA

≤ 10 mA
diseño por pin

3,3 V
máximo en una entrada

0,3 V
lo que le queda a un LED azul

No toleran 5 V

Un sensor o módulo de 5 V conectado directo a una entrada puede dañar el pin.

LEDs recomendados

Rojo, verde o amarillo con 220–330 Ω.
Azul y blanco casi no encienden a 3,3 V con resistencia.

Actividad: ¿dónde lo conecto?

A LED en GPIO18

B Pulsador en GPIO35 con INPUT_PULLUP

C LED en GPIO2

D Pulsador entre GPIO12 y 3,3 V

E Sensor que entrega 5 V en GPIO4

F LED en GPIO1

En parejas: ✓ o ✗ y una frase que lo justifique. Tienen 6 minutos.

2

Parte 2 · El programa que nunca termina

Cómo arranca y cómo vive un programa embebido

El visitante y el guardia nocturno

Programa de PC · el visitante

- Lo lanza el sistema operativo
- Hace su trabajo
- main() termina con return 0
- El SO recupera el control

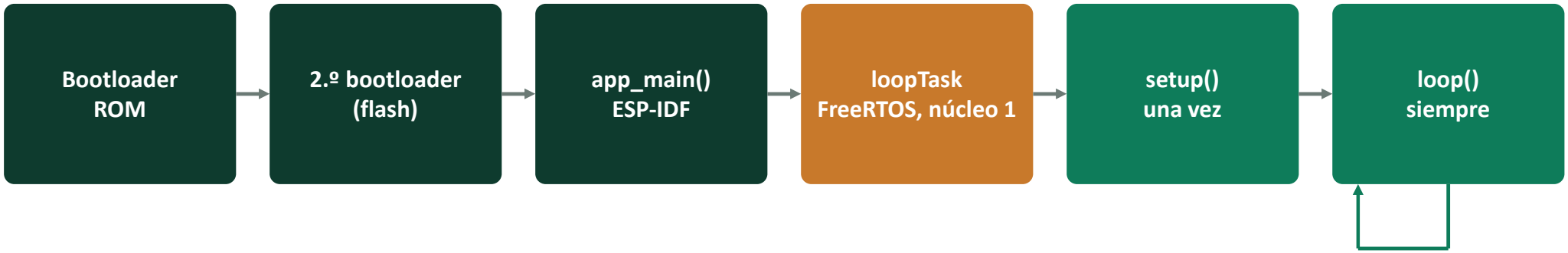
Programa embebido · el guardia

- Arranca con la alimentación
- Configura una vez
- Repite su ronda para siempre
- Solo termina si se corta la energía

En un microcontrolador no existe un "fin del programa".

Del reset a loop()

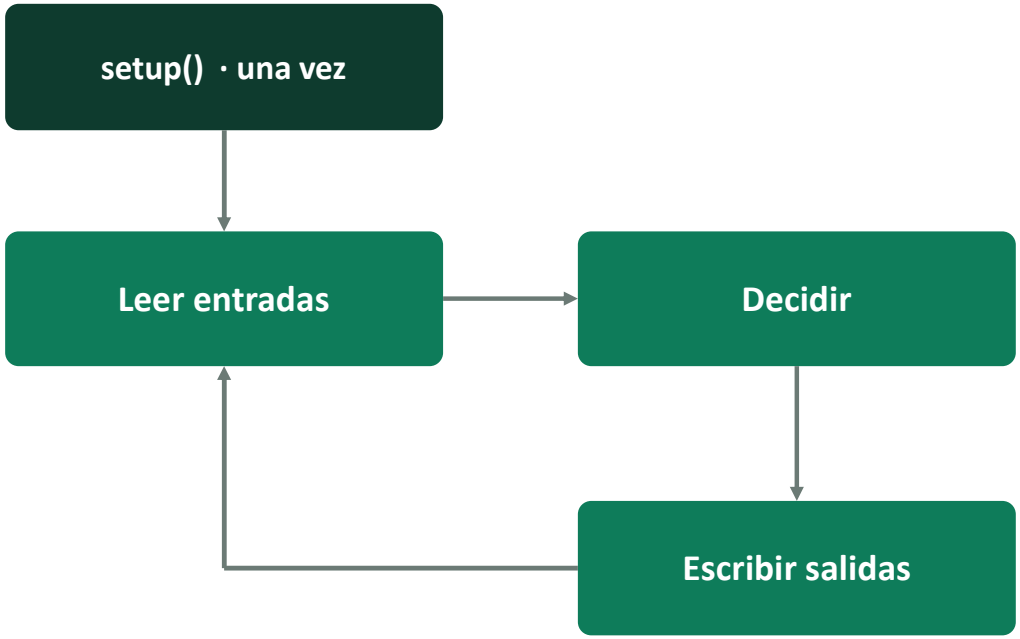
Aquí se leen los pines de strapping



Antes de setup(): las globales inicializadas se copian de la flash a la SRAM (como F() en la semana 1).

Todo sketch del ESP32 ya corre sobre FreeRTOS.
Lo estudiaremos en la Unidad 2.

El superlazo: setup() y loop()



loop() · para siempre

Lo que Arduino esconde (AVR):

```
int main() {
  init();       // timers, ADC...
  setup();
  for (;;) {
    loop();
  }
}
```

En el ESP32, loopTask hace lo mismo.

Predicción: ¿qué imprime en la 3.ª pasada?

```
int contadorGlobal = 0;

void setup() { Serial.begin(115200); }

void loop() {
  int contadorLocal = 0;
  static int contadorStatic = 0;

  contadorGlobal++; contadorLocal++; contadorStatic++;
  Serial.print("global=");  Serial.print(contadorGlobal);
  Serial.print(" local=");  Serial.print(contadorLocal);
  Serial.print(" static="); Serial.println(contadorStatic);
  delay(1000);
}
```

Línea 3:

```
global = ?
local  = ?
static = ?
```

Escriban su predicción antes de ejecutar en Wokwi.

Lo que cuesta un loop() vacío

240 MHz

girando aunque loop() no tenga nada que hacer

Activo

La CPU ejecuta instrucciones sin parar: decenas de mA.

Deep sleep

Casi todo se apaga y un temporizador lo despierta: del orden de µA.

Para sus proyectos: ¿el dispositivo necesita estar despierto todo el tiempo?

Cifras de orden de magnitud: verificar en el datasheet de Espressif.

3

Parte 3 · Manos a la herramienta

Arduino IDE + core Espressif 3.x

Instalar el core ESP32 en Arduino IDE 2.x

1 → 2 → 3 → 4

Preferencias

Agregar la URL del gestor de placas de Espressif

Gestor de placas

Instalar «esp32 by Espressif Systems» 3.x

Placa

Elegir «ESP32 Dev Module»

Puerto + driver

COMx o /dev/ttyUSB0 · driver CP210x o CH340

```
https://espressif.github.io/arduino-esp32/package_esp32_index.json
```

No es «Arduino ESP32 Boards». Instalen en casa: el core pesa varios cientos de MB.

Cuando el programa no sube

Síntoma	Causa probable	Solución
No aparece ningún puerto	Cable solo de carga o falta driver	Otro cable · driver CP210x/CH340
Wrong boot mode / Timed out	No entró en modo de carga	Mantener BOOT en «Connecting…»
Access denied	Puerto ocupado por otra ventana	Cerrar el otro Monitor Serie
Caracteres basura	Velocidad distinta	115200 en Serial.begin y Monitor
Brownout detector	Alimentación débil	Otro puerto USB o cable
No arranca con el circuito	Pin de strapping forzado	Mover a un pin seguro

Resumen

1 **Registros**

DDR/PORT/PIN → ENABLE / OUT (W1TS, W1TC) / IN

2 **Atomicidad**

W1TS y W1TC escriben sin leer: nada se pierde

3 **Pines**

Seguros · strapping · solo entrada · UART0 · 3,3 V

4 **Programa**

setup() una vez, loop() siempre, sobre FreeRTOS

Tarea para mañana

1	**Instalar**	Arduino IDE 2.x + core «esp32 by Espressif Systems» 3.x (en casa).
2	**Probar en Wokwi**	Blink con registros en el ESP32 DevKit (LED en GPIO16).
3	**Probar la placa**	Grupos con ESP32: cargar Blink con el LED de la placa.
4	**Leer**	Lectura del texto base indicada en el EVA.

Mañana: control de aprendizaje. Viernes: AP2 con ESP32 → traigan la placa y un cable USB de datos.